

DAR ES SALAAM INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER STUDIES

NTA LEVEL 8



BLOCKCHAIN-BASED NGO DONATION MANAGEMENT SYSTEM

MODULE NAME: PROJECT REALIZATION

MODULE CODE: COU 08204

COURSE: BENG22COE

CANDIDATE NAME: NASIBU Y. ISAKA

REGISTRATION NUMBER: 2106307225519

SUPERVISOR'S NAME: DR GUSTAPH SANGA

[PENDING: SUBMISSION DATE]

BLOCKCHAIN-BASED NGO DONATION MANAGEMENT SYSTEM

Changia

NASIBU Y. ISAKA

2106307225519

A project report submitted in partial fulfilment of the requirements for
the award of NTA Level 8 in the Department of Computer Studies of
the Dar es Salaam Institute of Technology.

DAR ES SALAAM INSTITUTE OF TECHNOLOGY

[PENDING: SUBMISSION DATE]

Declaration

I, **Nasibu Y. Isaka**, registration number **2106307225519**, declare that this report is my own original work and that it has not been presented, and will not be presented, to any other institution for a similar or any other award.

Signature: _____

Date: _____

Certification

The undersigned certifies that he has read and hereby recommends for acceptance by the Dar es Salaam Institute of Technology a project report titled **Blockchain-Based NGO Donation Management System**, submitted in partial fulfilment of the requirements for the award of NTA Level 8 in the Department of Computer Studies.

Dr Gustaph Sanga

Supervisor

Signature: _____

Date: _____

Dedication

This work is dedicated to my family, mentors and all people working to make charitable support more transparent and accountable.

Acknowledgements

I sincerely thank my supervisor, Dr Gustaph Sanga, for guidance throughout this project. I also appreciate the Department of Computer Studies, the people who contributed early questionnaire feedback, and my family and colleagues for their support during design, implementation and testing.

Abstract

NGO donation systems need to make contribution and fund-release records more visible without making ordinary donors adopt cryptocurrency. This project designed and implemented ChangiaTanzania, a blockchain-based NGO donation management system that combines familiar web and local-payment workflows with privacy-preserving Ethereum proof recording. An iterative Agile approach was used, informed by retained questionnaire themes, document review, source inspection and technical testing. The system implements donor, fundraiser and administrator roles; campaign management; beneficiary verification; payment-session handling; idempotent payment callback finalisation; receipts; donation history; notifications; dashboards; audit logs; exports; controlled disbursements; public receipt verification; and an integrated risk-review module. PostgreSQL is the operational source of truth. After a verified payment or completed disbursement, the back-end records a deterministic proof hash through a smart contract, while public verification omits donor identity and payment references. The implementation demonstrates that blockchain can serve as a narrow tamper-evident proof layer alongside conventional application controls rather than replacing payment or operational systems. Tests cover key authentication, payment, proof, campaign-review, beneficiary, disbursement-control and risk-analysis scenarios. The report records two material limitations: individual response-level questionnaire rows were unavailable, so only the aggregated 26-respondent summary (Appendix A) is used, as directional rather than inferential evidence; and production payment credentials, infrastructure hardening, load testing and independent user acceptance testing remain required. The risk-review module is feature-flagged off until its migration is applied and it is deliberately enabled; the on-chain escrow contract remains future work.

Table of Contents

Declaration	i
Certification	ii
Dedication	iii
Acknowledgements	iv
Abstract	v
List of Figures	xi
List of Tables	xii
List of Abbreviations	xiii
List of Appendices	xiv
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Problem Statement	1
1.3 Aim of the Project	2
1.4 Specific Objectives	2
1.6 Scope of the Project	2
1.7 Significance of the Project	3
1.8 Limitations	3
CHAPTER TWO: LITERATURE REVIEW	4

2.1 Introduction	4
2.2 Theoretical Literature Review	4
2.3 Empirical Literature Review	5
2.4 Existing Systems Review	5
2.5 Project Research Gap	5
2.6 Proposed System	5
2.7 Conceptual Framework	6
2.8 Strengths of the Proposed System	6
CHAPTER THREE: PROJECT RESEARCH METHODOLOGY	7
3.1 Introduction	7
3.2 Research Design	7
3.3 Study Area	7
3.4 Target Population and their Categories	8
3.6 Sample Size	8
3.7 Data Collection Methods	8
3.8 Data Collection Instruments	8
3.9 Data Required for Each Objective	8
3.10 Ethical Considerations	9
3.11 Data Analysis Methods	9
3.12 Development Methodology	9
3.13 Software and Hardware Used	10
3.14 Summary	10
CHAPTER FOUR: DATA ANALYSIS	11
4.1 Introduction	11
4.2 Respondent Demographics	11
4.3 Analysis of Collected Data	11
4.4 Interpretation of Findings	12
4.5 Identified Problems	12
4.6 Functional Requirements Identified	12
4.7 Non-functional Requirements Identified	12
4.8 Summary	13

CHAPTER FIVE: SYSTEM REQUIREMENT SPECIFICATION	14
5.1 Introduction	14
5.2 Existing System Analysis	14
5.3 Proposed System	14
5.4 Functional Requirements	14
5.5 Non-functional Requirements	16
5.6 User Requirements	17
5.7 System Requirements	17
5.8 Feasibility Study	17
5.9 Software Requirement Specification	17
 CHAPTER SIX: SYSTEM DESIGN	 18
6.1 Introduction	18
6.2 System Architecture	18
6.3 Data Modeling	19
6.3.1 Context Diagram	19
Data Flow Diagrams	19
6.4.1 Database Design	19
6.5 Process Modeling	20
6.5.1 UML Models	21
6.7 User Interface Design	23
6.9 Machine Learning and Dataset Design	23
6.9.1 Algorithms of the Working System	24
6.10 Flowcharts of the Working System	24
6.11 Security Design	24
 CHAPTER SEVEN: SYSTEM IMPLEMENTATION, TESTING AND RE-	 25
SULTS	
7.1 Introduction	25
7.2 Development Environment	25
7.3 Software Tools Used	25
7.4 Implementation of Modules	25
7.5 Testing Strategy	26

7.6 Test Cases	27
7.7 Results	27
7.8 Discussion of Results	28
7.9 System Screenshots	28
CHAPTER EIGHT: CONCLUSION AND RECOMMENDATIONS	29
8.1 Summary	29
8.2 Achievement of Objectives	29
8.3 Conclusion	29
8.4 Recommendations	30
8.5 Future Work	30
REFERENCES	31
APPENDICES	32
Appendix A: Questionnaire	32
A.1 Instrument	32
A.2 Aggregated Responses (n = 26)	33
Appendix B: Document Review Matrix	34
Appendix C: Source Code (selected excerpts)	37
C.1 Deterministic Proof Hash	37
C.2 Idempotent Payment Callback	38
C.3 Dual-Approval Disbursement Rule	39
Appendix D: User Manual	40
D.1 Donor	40
D.2 Fundraiser	40
D.3 Administrator	41
Appendix E: Installation Guide	41
E.1 Local Development	41
E.2 Production Deployment (summary)	42
Appendix F: Test Cases	43
F.1 API and Workflow Test Cases	43
F.2 Smart Contract Unit Tests	45
F.3 Manual and Responsive Testing Checklist	46

Appendix G: Gantt Chart	46
Appendix H: Budget	48
Appendix I: Additional Screenshots	48
Appendix J: Full Entity Relationship Diagram	48

List of Figures

6.1	Figure 6.1: System architecture, layered view	18
6.2	Figure 6.2: Context diagram, DFD level 0	19
6.3	Figure 6.3: Data flow diagram, level 1	19
6.4	Figure 6.4: Entity relationship diagram (core entities)	20
6.5	Figure 6.5: Class diagram, backend service layering	21
6.6	Figure 6.6: Sequence diagram, donation and proof recording	22
6.7	Figure 6.7: Activity diagram, disbursement dual approval	22
6.8	Figure 6.8: Deployment diagram	23
8.1	Figure G.1: Project development timeline	47
8.2	Figure J.1: Full entity relationship diagram, all 14 tables	49

List of Tables

3.1	Table 3.1: Data required for each research objective	9
4.1	Table 4.1: Retained questionnaire themes mapped to ChangiaTanzania's response	11
5.1	Table 5.1: Functional requirements	15
5.2	Table 5.2: Non-functional requirements	16
7.1	Table 7.1: Implementation of modules	26
7.2	Table 7.2: Test case summary (full log in Appendix F)	27
8.1	Table A.1: Questionnaire instrument	32
8.2	Table A.2: Aggregated response summary	33
8.3	Table B.1: Document review matrix	34
8.4	Table E.1: Production hosting topology	42
8.5	Table F.1: API and workflow test cases	43
8.6	Table F.2: Smart contract unit tests	45
8.7	Table F.3: Manual and responsive testing checklist	46
8.8	Table G.1: Development phases by date	47
8.9	Table H.1: Project budget	48

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
CSV	Comma Separated Values
DFD	Data Flow Diagram
DIT	Dar es Salaam Institute of Technology
ERD	Entity Relationship Diagram
FR	Functional Requirement
JWT	JSON Web Token
NFR	Non-functional Requirement
NGO	Non-Governmental Organization
NTA	National Technical Award
ORM	Object Relational Mapping
RBAC	Role Based Access Control
RPC	Remote Procedure Call
SDLC	Software Development Life Cycle
SRS	Software Requirement Specification
TZS	Tanzanian Shilling
UAT	User Acceptance Testing
UML	Unified Modeling Language

List of Appendices

Appendix	Title
A	Questionnaire
B	Document Review Matrix
C	Source Code (selected excerpts)
D	User Manual
E	Installation Guide
F	Test Cases
G	Gantt Chart
H	Budget
I	Additional Screenshots
J	Full Entity Relationship Diagram

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

NGOs depend on credible stewardship of donations. Digital payment confirmation does not, by itself, show a donor whether a contribution was recorded correctly or whether later disbursement followed agreed controls. Tanzania is an appropriate setting for a locally usable solution because mobile money has expanded access to financial services (World Bank, 2017). Blockchain can preserve a verifiable record of material events, although it does not remove governance and privacy responsibilities (Howson, 2021; Sung et al., 2023).

ChangiaTanzania is a web-based donation management system. PostgreSQL is the operational source of truth; Ethereum stores minimal cryptographic proofs after verified payment or completed disbursement. Donors need no cryptocurrency wallet. The system supports donors, verified fundraisers and administrators, with campaign review, beneficiary verification, controlled disbursements, public receipt verification, reporting, notifications and audit records.

1.2 Problem Statement

Conventional donation workflows often separate payment collection, campaign administration and utilisation reporting. A donor may receive an acknowledgement but lack an independent way to check the integrity of a later record or trace a completed disbursement. Cryptocurrency-first charity platforms can improve traceability but do not suit users who normally use local mobile-money services. The problem addressed is the absence of a

practical, role-controlled platform that combines familiar Tanzanian payment rails with privacy-preserving, verifiable transaction proofs.

1.3 Aim of the Project

To design and implement a blockchain-based NGO donation management system that improves transparency, accountability and secure administration of campaigns, donations and beneficiary disbursements while retaining familiar local payment workflows.

1.4 Specific Objectives

1. To implement secure role-based access for donors, fundraisers and administrators.
2. To implement campaign, beneficiary and donation management with payment confirmation and downloadable receipts.
3. To record privacy-preserving proofs of completed donations and disbursements on Ethereum and provide public receipt verification.
4. To implement accountable fund release through beneficiary verification, balance checks, separation of duties, audit logging and reporting.
5. To implement decision support through explainable risk assessment, review queues, notifications and dashboards.

1.6 Scope of the Project

The project delivers a responsive web system for registered NGOs and community fundraisers: account security, campaigns, beneficiaries, donations, payment-provider abstraction, receipts, notifications, reports, audit logs, public blockchain-proof lookup and an administrator risk-review queue. The hybrid risk assessment is integrated into the codebase but feature-flagged off until its database migration is applied and it is deliberately enabled. The system does not process cryptocurrency payments, store PII on-chain, provide a native mobile application, or claim production-mainnet use.

1.7 Significance of the Project

Donors receive receipts, a personal history and a public verification route that reveals no donor identity. NGOs and fundraisers receive structured campaign, beneficiary and reporting workflows. Administrators receive separation-of-duties controls, audit records and exports. The work demonstrates a practical integration of conventional payment services, a relational application and public-chain proof without forcing users to understand wallets or digital assets.

1.8 Limitations

The current public-chain configuration is a test environment; live merchant credentials and production chain custody are outside this report. Local uploads should be replaced by durable object storage before production. Cloud free-tier cold starts can affect initial latency. The response-level questionnaire export is unavailable, so this report presents only its documented themes and does not invent quantitative results.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the concepts and prior systems that inform ChangiaTanzania, distinguishing a blockchain proof ledger from the operational database.

2.2 Theoretical Literature Review

Blockchain is a replicated ledger whose records are cryptographically linked. In this project its role is deliberately narrow: a hash of a completed event can later be reconciled without publishing personal data. Donation-traceability research identifies immutable event records and end-to-end traceability as a response to uncertainty over charitable funds (Alabdulkarim et al., 2022). Sung et al. found that transparency, immutability and efficiency features of a blockchain donation system improved perceived nonprofit trustworthiness (Sung et al., 2023).

Technology is not a substitute for governance. Howson identifies trade-offs between transparency, power and beneficiary interests in crypto-giving (Howson, 2021). ChangiaTanzania therefore holds personal and operational data off-chain, exposes only a restricted verification result, and uses role separation and audit logs for governance.

2.3 Empirical Literature Review

Charity frameworks commonly emphasise traceability, notification and auditable histories (Alabdulkarim et al., 2022; Muneeb et al., 2020). Their cryptocurrency orientation creates an adoption mismatch for a mobile-money context. World Bank analysis describes Tanzania’s mobile-money expansion as a major contributor to financial inclusion (World Bank, 2017). ChangiaTanzania consequently verifies a fiat payment first, then creates a database donation and records its proof asynchronously.

2.4 Existing Systems Review

Generic crowdfunding platforms demonstrate campaign pages, online collection and receipts, but their operational controls are not necessarily visible to an individual donor. Blockchain charity frameworks offer stronger traceability but frequently assume cryptocurrency payment and wallet use (Muneeb et al., 2020). Neither pattern alone combines local fiat-payment abstraction, beneficiary verification, campaign-owner controls, cumulative self-release limits and privacy-preserving public proof lookup.

2.5 Project Research Gap

The gap is an implementable accountability workflow that does not move ordinary donors into cryptocurrency. ChangiaTanzania keeps donation value off-chain, makes PostgreSQL the authoritative operational record, and anchors a deterministic proof hash only after verified completion. Its fund-release workflow adds governance that a proof alone cannot supply.

2.6 Proposed System

A donor browses an active campaign, starts a payment session and receives a receipt after a verified callback. The transaction is committed atomically, totals are updated and an asynchronous service records proof. Fundraisers own their campaigns but cannot publish without approval; beneficiaries remain administrator-verified; disbursements are constrained by available balance and approval rules.

2.7 Conceptual Framework

Inputs are credentials, campaign data, payment callbacks and payout requests. Processing validates input, applies roles and business rules, writes PostgreSQL transactions, produces notifications and queues proof recording. Outputs are receipts, dashboards, exports, audit entries and verification results. Public verification, administrative reviews and risk assessments form the feedback loop.

2.8 Strengths of the Proposed System

Key strengths are familiar payment journeys, operational data separated from immutable proof data, role-based governance, idempotent payment finalisation, public verification without PII disclosure and provider abstraction. A pending chain write never blocks a successfully verified payment and can later be reconciled.

CHAPTER THREE: PROJECT RESEARCH METHODOLOGY

3.1 Introduction

The project used applied systems development: early user-problem evidence was translated into requirements, then delivered through iterative implementation and technical verification.

3.2 Research Design

An iterative Agile approach was used. The system was built in vertical stages: public campaigns, authentication, donations, blockchain proof, administration, community fundraising and payment/deployment improvements. A Google Forms questionnaire provided problem and preference themes; document review and observation supplemented it.

3.3 Study Area

The context was Tanzania. The questionnaire was shared through Tanzania Red Cross Society cooperation chat groups and among students, focusing on donation trust, tracking and payment preferences.

3.4 Target Population and their Categories

The intended population was potential donors, NGO stakeholders and students. Administrative workflows were analysed from the perspectives of a neutral NGO operator and a community fundraiser.

3.6 Sample Size

The Google Form received 26 responses. Individual response rows and collection dates were not retained, only the aggregated per-question summary reproduced in Appendix A, so the sample supports directional themes rather than inferential findings. A future evaluation should retain anonymised response data, consent information and collection dates.

3.7 Data Collection Methods

The questionnaire collected early user views. Document review examined requirements, platform documentation and donation-control literature. Observation examined expected platform behaviour and the developed application. Interviews were not used within the available project time.

3.8 Data Collection Instruments

The instruments were a structured Google Forms questionnaire and document-review matrix. Questionnaire themes concerned trust, tracking, transparency, blockchain verification and donation management.

3.9 Data Required for Each Objective

Table 3.1: Data required for each research objective

Objective	Evidence	Method
Secure access	roles, sessions and access rules	source review and API tests
Donation workflow	payment, receipt and history	E2E tests
Proof	hash, transaction and lookup	contract/API tests
Accountable payout	beneficiary, balance and approval controls	business rules and E2E tests
Decision support	risk signals and review path	integrated risk-module code/tests

3.10 Ethical Considerations

Participation was voluntary and findings are reported only as aggregate themes. Public verification returns campaign, amount, date and transaction reference but no donor identity, email or payment reference. On-chain data is a proof hash rather than personal data.

3.11 Data Analysis Methods

The retained questionnaire findings were analysed thematically. Technical evidence was analysed through requirements traceability, source inspection, automated test results and manual workflow verification. No quantitative result is presented without its source data.

3.12 Development Methodology

Agile development was appropriate because payments, blockchain and role controls have interdependencies. Each stage added a usable capability and was checked through linting, type checking, builds and targeted end-to-end tests. Git history preserves the iteration record.

3.13 Software and Hardware Used

React, TypeScript, Vite and Tailwind CSS implement the client. Node.js, Express, Drizzle ORM and PostgreSQL implement the server. Solidity, Hardhat and Ethers implement blockchain proof. Development used Windows 11, VS Code, PowerShell, Git and npm; deployment targets are Vercel, Render and Neon PostgreSQL.

3.14 Summary

The methodology combines user-problem themes with iterative implementation and reproducible technical verification, while explicitly acknowledging missing response-level survey data.

CHAPTER FOUR: DATA ANALYSIS

4.1 Introduction

This chapter reports evidence available without overstating it: retained questionnaire themes, requirements analysis and reproducible technical tests.

4.2 Respondent Demographics

The questionnaire was shared through Tanzania Red Cross Society cooperation chat groups and a student population (26 respondents). Individual demographic fields were not retained, so per-respondent breakdowns (age, gender, occupation) are not reproduced; the retained artefact is the aggregated response summary reproduced in Appendix A.

4.3 Analysis of Collected Data

The retained questionnaire summary identified uncertain trust in existing NGO donation systems, difficulty tracking a donation after payment, demand for transparent tracking, support for blockchain-backed records, preference for mobile money, and demand for improved donation management. These are design inputs, not generalisable statistical estimates.

Table 4.1: Retained questionnaire themes mapped to ChangiaTanzania's response

Finding	ChangiaTanzania response
Uncertain trust	Public receipt verification and audit records

Finding	ChangiaTanzania response
Tracking is difficult	Receipt, history, campaign progress and proof status
Transparency is expected	Verifiable donation and disbursement proofs
Local payment is preferred	Tanzanian payment-provider abstraction
Management needs improvement	Campaign, beneficiary, payout and reporting modules

4.4 Interpretation of Findings

The evidence supports an accountability architecture rather than a payment page alone. It explains the system's payment-to-proof workflow, limited public verification view, and administrative controls around beneficiaries and payouts. It does not establish that blockchain itself causes trust.

4.5 Identified Problems

The analysis identified insufficient post-donation visibility, limited traceability of fund use, reliance on privileged administration and mismatch between cryptocurrency-first platforms and local payment habits.

4.6 Functional Requirements Identified

The resulting functional requirements are specified and traced in Section 5.4: access, campaigns, payments, history, proofs, beneficiaries, disbursements, reports, notifications and risk review.

4.7 Non-functional Requirements Identified

Security, reliability, usability, maintainability, availability and scalability were identified as essential qualities. Section 5.5 converts them into testable acceptance targets.

4.8 Summary

The available evidence supports a transparent, mobile-money-compatible workflow. The aggregated response summary (Appendix A) is directional evidence from a convenience sample of 26 respondents, not a statistically representative survey; it is used to motivate design themes rather than to support inferential claims.

CHAPTER FIVE: SYSTEM REQUIREMENT SPECIFICATION

5.1 Introduction

This chapter specifies what ChangiaTanzania must do and its operational constraints.

5.2 Existing System Analysis

Manual records and generic online fundraising leave a weak link between payment acknowledgement, fund use and independently checkable evidence. Centralised data also needs governance controls to prevent a single user from creating and approving exceptional payouts.

5.3 Proposed System

The proposed system combines a public interface, protected role workspaces, payment-provider adapters, PostgreSQL operations, Ethereum proof recording and report exports. It addresses visibility with receipts and proof lookup, and governance with beneficiary verification, approval rules and immutable audit logs.

5.4 Functional Requirements

Table 5.1: Functional requirements

ID	The system shall	Priority
FR-01	register, authenticate and manage secure user sessions	High
FR-02	enforce donor, fundraiser and administrator permissions	High
FR-03	list, search and display active campaigns	High
FR-04	let authorised owners create, edit, submit and archive campaigns	High
FR-05	create payment sessions and finalise verified callbacks idempotently	High
FR-06	issue receipts and show each donor's donation history	High
FR-07	create and publicly verify donation and disbursement proofs	High
FR-08	manage beneficiaries and restrict payouts to verified beneficiaries	High
FR-09	enforce balance, threshold and self-approval rules for disbursements	High
FR-10	notify users and provide dashboards, audit records and exports	Medium

ID	The system shall	Priority
FR-11	provide an administrator risk-review API and queue for unusual completed donations	Medium

5.5 Non-functional Requirements

Table 5.2: Non-functional requirements

Quality	Acceptance target / evidence
Security	bcrypt passwords, JWT access, rotating httpOnly refresh token, RBAC, validation and rate limits
Reliability	one atomic payment finalisation; duplicate callbacks do not double-credit
Privacy	public lookup exposes no donor PII or payment reference; no PII on-chain
Maintainability	strict TypeScript, layered API and provider interfaces
Usability	responsive client with loading, empty, error and success states
Availability	a proof-write failure does not discard a verified payment
Scalability	PostgreSQL pagination and constraints; production load testing remains required

5.6 User Requirements

Donors need browsing, donation, receipts, history and verification. Fundraisers need controlled ownership of campaigns, beneficiaries and permitted payouts. Administrators need oversight, review, verification, reports, audit logs and user controls.

5.7 System Requirements

The client requires a modern browser and internet connection. Server deployment requires Node.js 20+, PostgreSQL and protected secrets. Blockchain proof requires an Ethereum RPC endpoint, backend wallet and contract address. Development uses React, Vite, Express, Drizzle, Hardhat and Solidity.

5.8 Feasibility Study

The solution uses mature tooling and preserves familiar fiat payments. Development and test deployment can use low-cost tiers and test ETH, but production must budget for gateway fees, storage, monitoring and gas. A production NGO must meet applicable payment, data-protection and charity regulations. The staged Git history demonstrates schedule feasibility.

5.9 Software Requirement Specification

The human actors are Donor, Fundraiser and Administrator; the payment provider and Ethereum network are external systems. The functional requirements above are the formal SRS baseline and are traced in Chapter Seven.

CHAPTER SIX: SYSTEM DESIGN

6.1 Introduction

Chapter Five states what the system must do; this chapter explains how it is structured to do it. The design uses a layered web architecture, relational data model, Ethereum proof registry and documented process controls.

6.2 System Architecture

The React client communicates with the Express API. Routes delegate to controllers, services and repositories; the repository layer persists operational state to PostgreSQL. Payment providers and Ethereum are external integrations behind service boundaries, ensuring that a provider or proof-write failure does not corrupt the completed donation record.

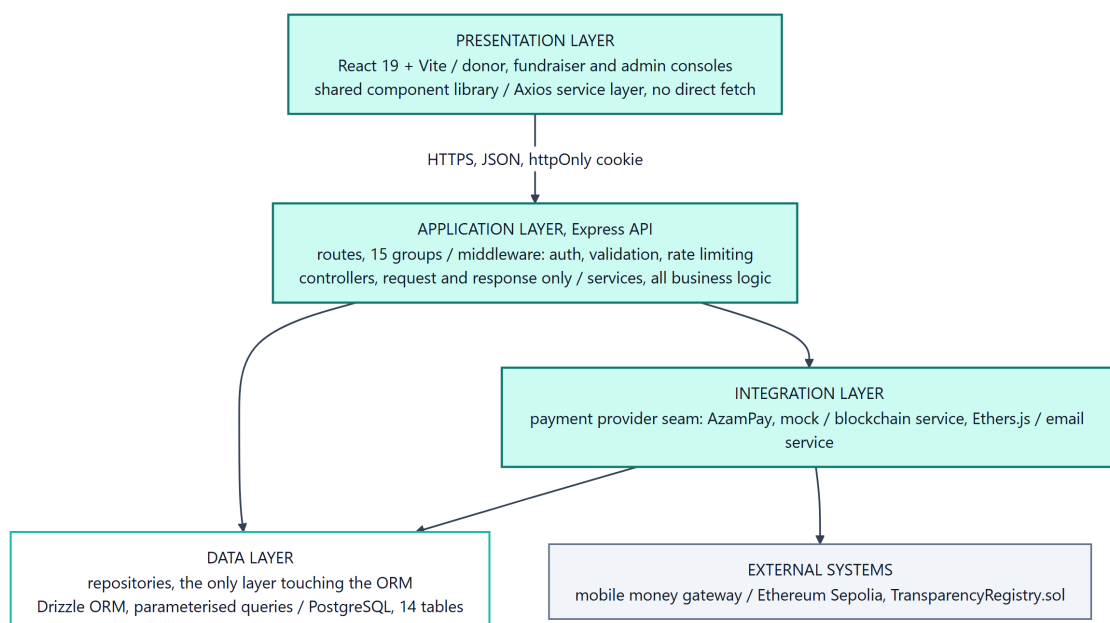


Figure 6.1: System architecture, layered view

6.3 Data Modeling

6.3.1 Context Diagram

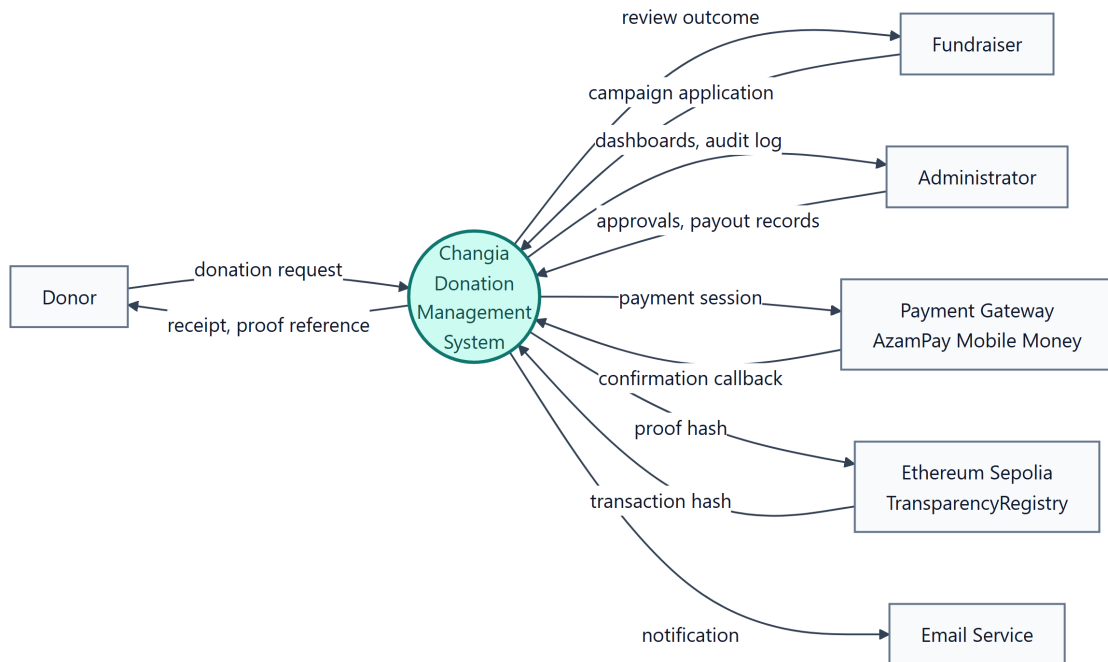


Figure 6.2: Context diagram, DFD level 0

Data Flow Diagrams

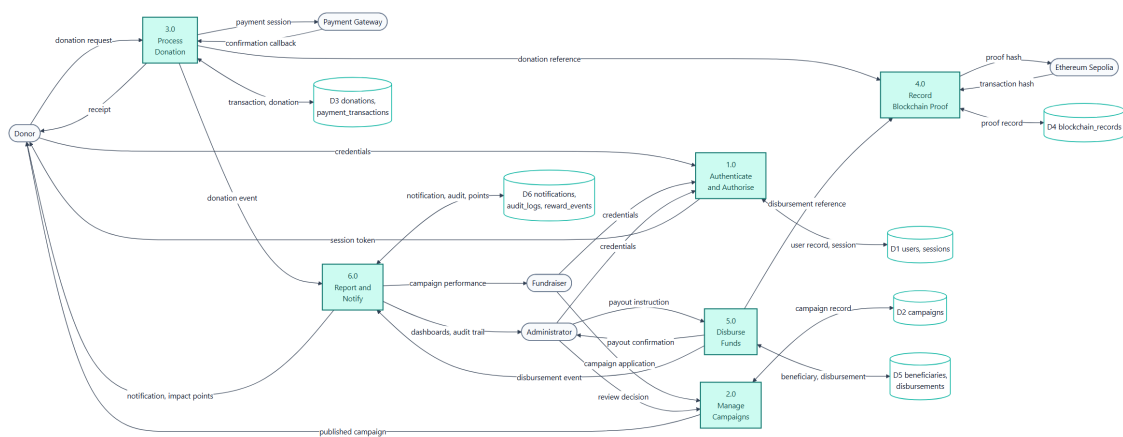


Figure 6.3: Data flow diagram, level 1

6.4.1 Database Design

The relational model holds users, sessions, password resets, campaigns, donations, payment transactions, beneficiaries, disbursements, approvals, blockchain records, notifica-

tions, audit logs, rewards and fundraiser applications. IDs are BIGINT values and money is stored in whole Tanzanian shillings. Donations, disbursements and blockchain records are immutable business records; mutable entities use controlled status changes or soft deletion.

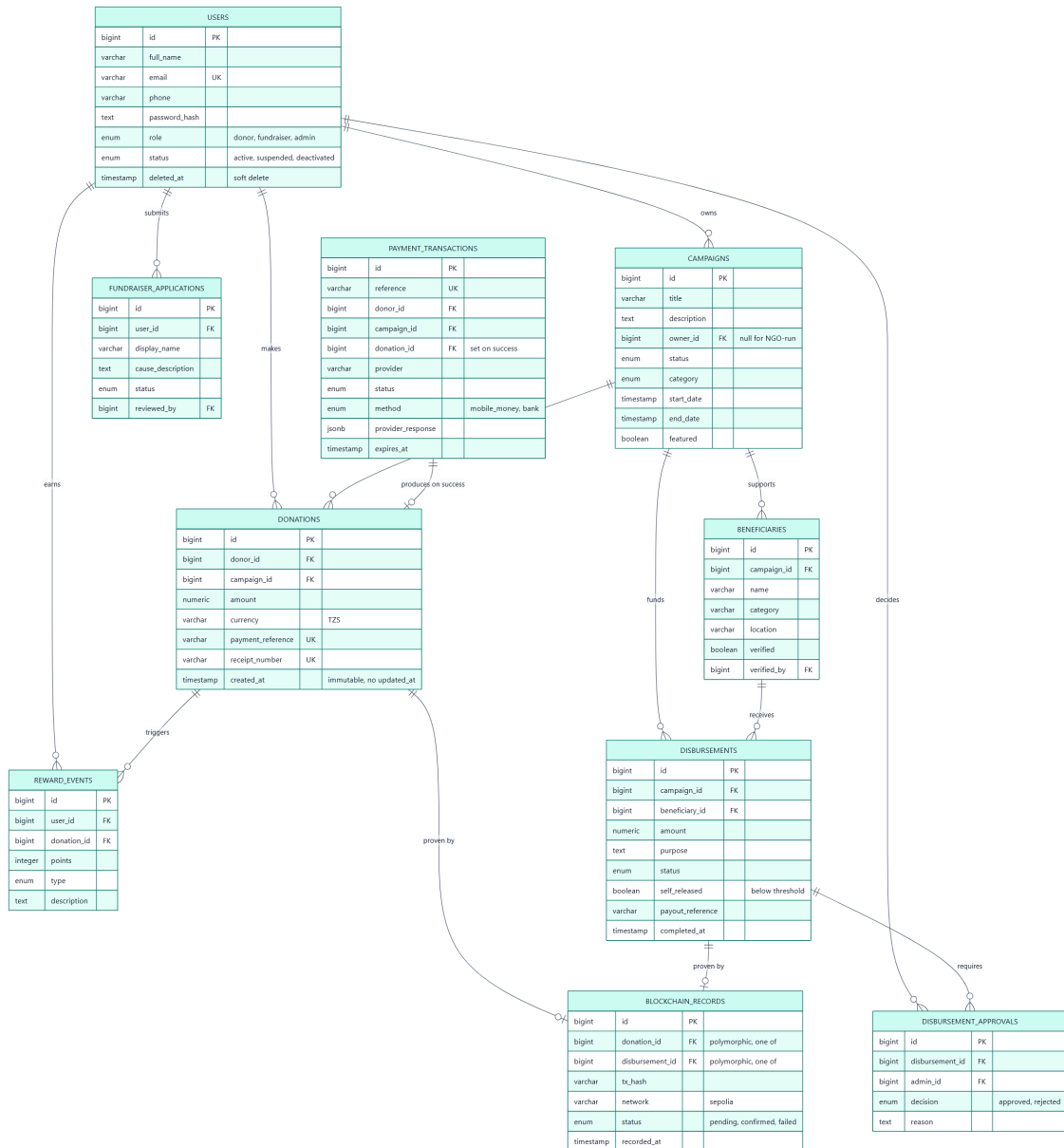


Figure 6.4: Entity relationship diagram (core entities)

6.5 Process Modeling

6.5.1 UML Models

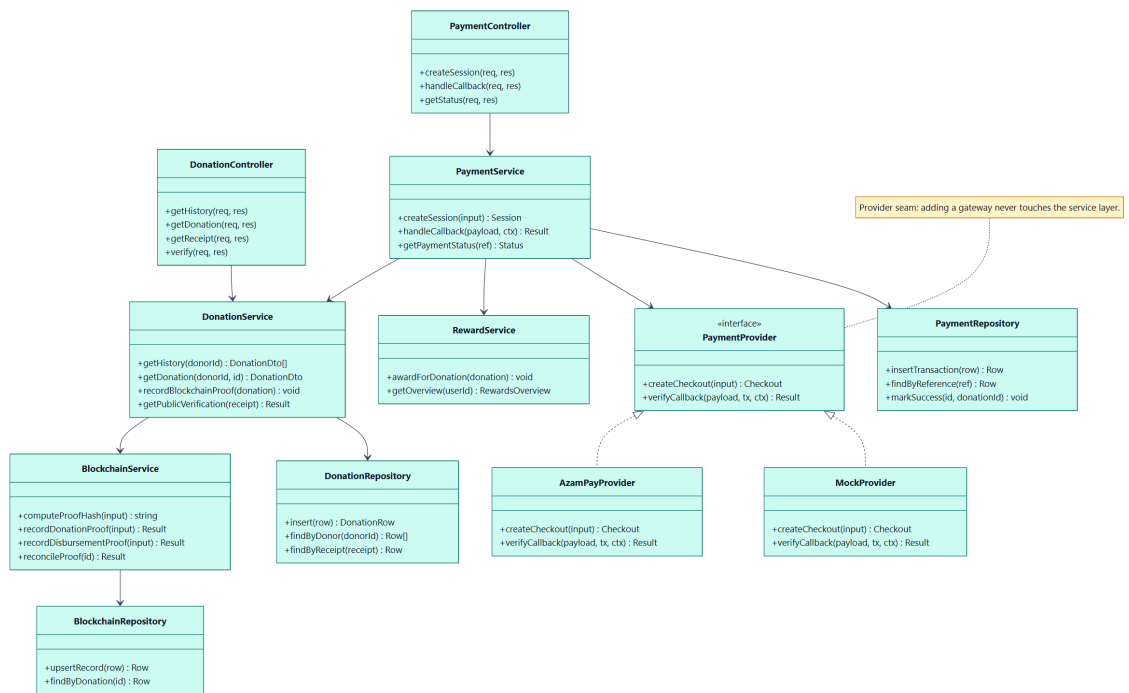


Figure 6.5: Class diagram, backend service layering

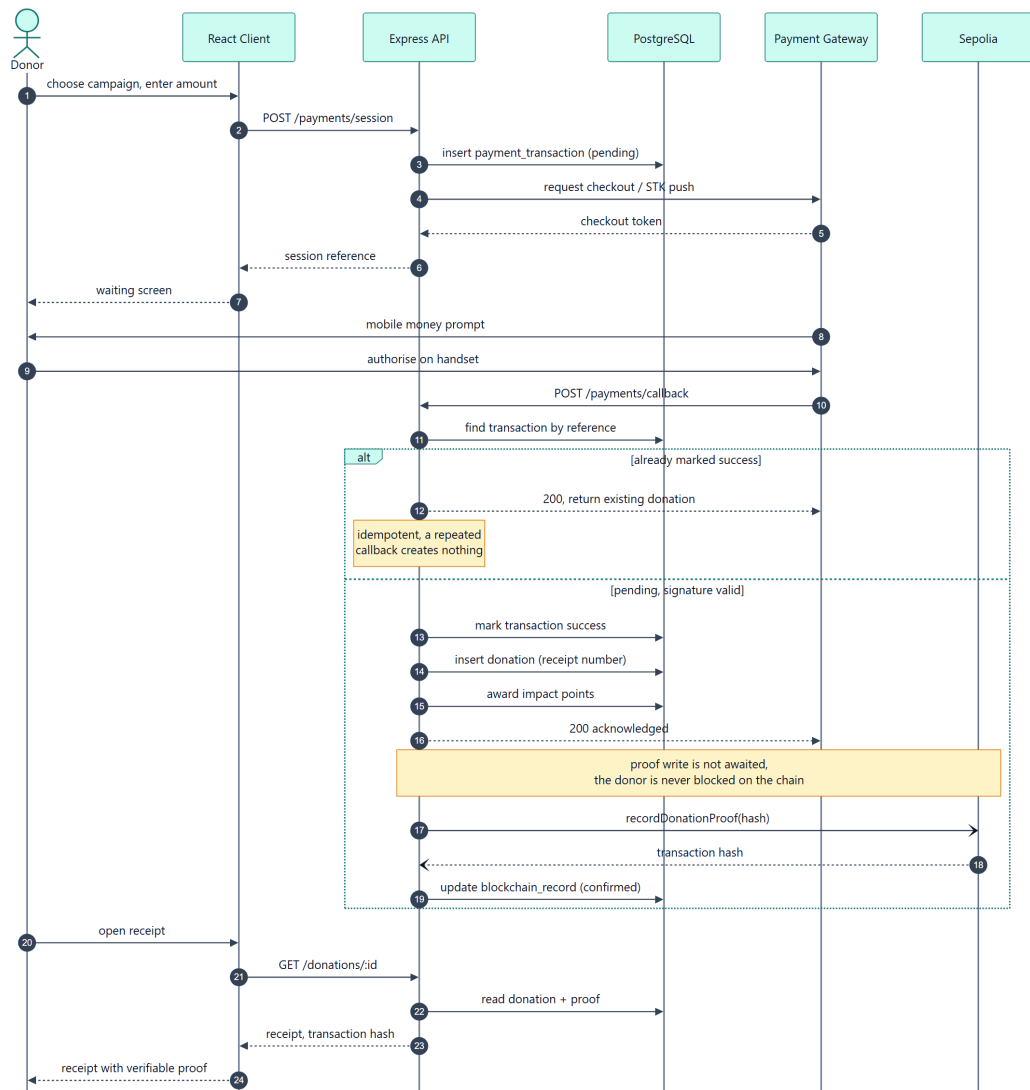


Figure 6.6: Sequence diagram, donation and proof recording

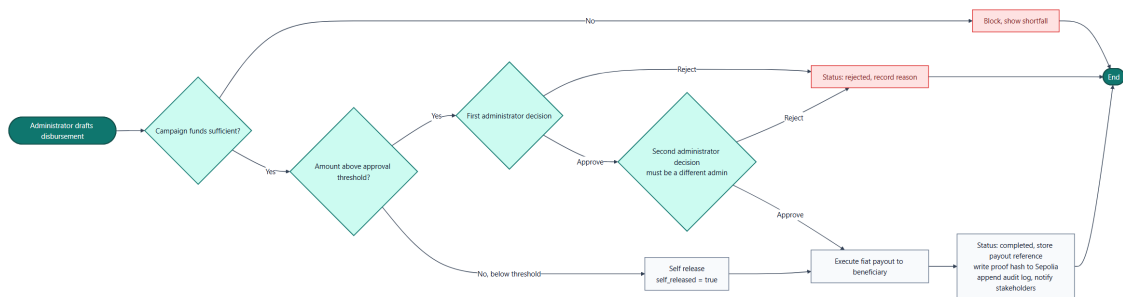


Figure 6.7: Activity diagram, disbursement dual approval

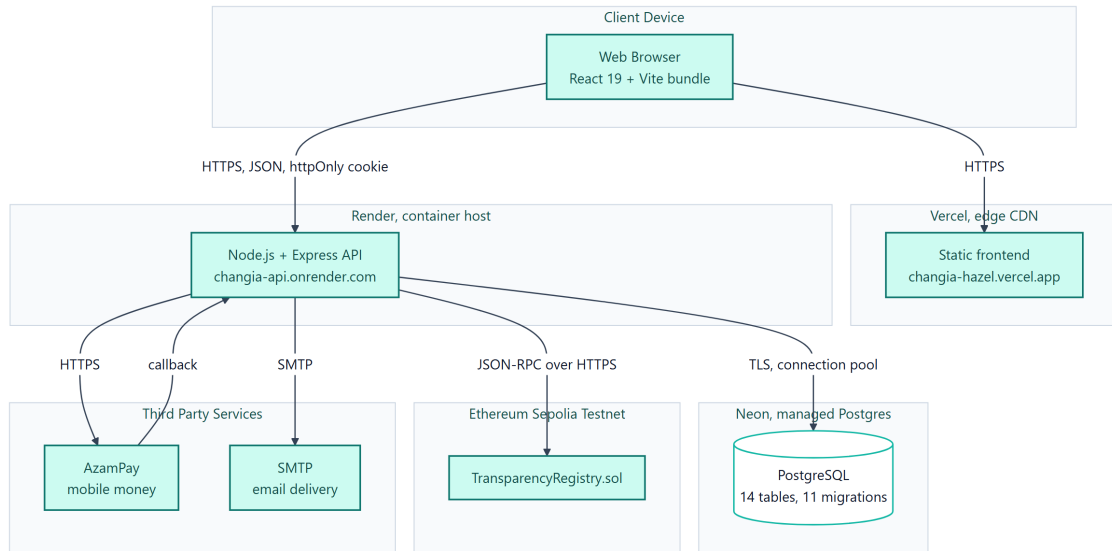


Figure 6.8: Deployment diagram

6.7 User Interface Design

The public interface prioritises campaign discovery, understandable donation controls and proof lookup. Protected interfaces are role-specific: donors see history and receipts; fundraisers see their owned campaigns and permitted actions; administrators receive operational navigation, reviews, audit records and reports. Each page includes explicit loading, empty, error and success states.

6.9 Machine Learning and Dataset Design

The integrated risk module implements an explainable hybrid risk assessment. Rules produce human-readable signals while an Isolation Forest identifies unusual combinations of transaction features. The model is unsupervised because labelled fraud history is unavailable; it is only fitted after sufficient donation history and is re-fitted on a schedule. Reviews can later create the labels needed for supervised evaluation. The module is feature-flagged off by default, so it does not affect payment completion until the migration is applied and the flag is deliberately enabled.

6.9.1 Algorithms of the Working System

For a completed donation, the service combines donation identifier, campaign identifier, amount, receipt number, payment reference and timestamp into a SHA-256 proof hash. The database finalises the donation before attempting the chain write. For payouts, the service checks campaign ownership, verified beneficiary, available balance and threshold/initiator rules before completing a disbursement.

6.10 Flowcharts of the Working System

The payment workflow is: validate request; create provider session; verify callback; atomically create donation and payment completion; update campaign; schedule proof; notify donor. The disbursement workflow validates beneficiary and balance, then either completes under permitted rules or waits for a separate administrator decision.

6.11 Security Design

This design implements the Security NFR in Section 5.5 with bcrypt password hashing, short-lived JWT access tokens, rotating httpOnly refresh tokens, RBAC, account lockout, rate limits, validation, parameterised database access, audit logging and safe upload names. The proof registry stores hashes rather than donor data, and public verification omits donor identity and payment references.

CHAPTER SEVEN: SYSTEM IMPLEMENTATION, TESTING AND RESULTS

7.1 Introduction

ChangiaTanzania was implemented as a TypeScript web application with a separately tested Solidity proof contract.

7.2 Development Environment

Development used Windows 11, Visual Studio Code, PowerShell, Git and npm. The client uses React/Vite; the API uses Express; PostgreSQL is accessed through Drizzle; contracts are built with Hardhat.

7.3 Software Tools Used

React, TypeScript, Vite, Tailwind CSS, shadcn/ui, Node.js, Express, PostgreSQL, Drizzle ORM, Zod, Ethers, Solidity, Hardhat, ESLint and Git were used. CSV, Excel and PDF libraries support exports.

7.4 Implementation of Modules

Table 7.1: Implementation of modules

Module	Capability	Requirements
Identity	registration, login, reset, sessions and RBAC	FR-01, FR-02
Campaigns	public browsing and owner/admin management	FR-03, FR-04
Donations	payment sessions, callback finalisation, receipts, history	FR-05, FR-06
Proofs	deterministic hashes, registry writes and public verification	FR-07
Payouts	beneficiary verification, balance and approvals	FR-08, FR-09
Operations	notifications, dashboard, audit log and exports	FR-10
Risk review	explainable rules, statistical signals, Isolation Forest and review queue	FR-11

Payment finalisation is idempotent and precedes asynchronous proof recording; a chain failure cannot lose a verified donation and later verification can reconcile pending proof.

7.5 Testing Strategy

Testing combines smart-contract unit tests, API end-to-end tests, client build/lint/type checks and manual browser workflows. Contract tests cover proof registration and duplicate rejection. API tests cover registration, authorisation, payment finalisation, proof lookup, campaign review, beneficiary verification, disbursement controls and reports. The integrated risk module adds analytics and Isolation Forest tests. Independent UAT and production load testing remain future work.

7.6 Test Cases

Table 7.2: Test case summary (full log in Appendix F)

ID	Description	Input	Expected	
			Result	Status
TC-01	Registration	Valid credentials	Account/session created	Pass
TC-02	Duplicate callback	Same reference twice	One donation and credit	Pass
TC-03	Public proof lookup	Receipt number	Status without PII	Pass
TC-04	Campaign review	Fundraiser submission	Not public before approval	Pass
TC-05	Beneficiary control	Unverified beneficiary	Payout rejected	Pass
TC-06	Separation of duties	Initiator approves own payout	Approval rejected	Pass
TC-07	Balance control	Excess payout	Request rejected	Pass
TC-08	Duplicate proof	Existing identifier	Contract rejects it	Pass

7.7 Results

The tested implementation satisfies the core workflow: donor payment, receipt, history and verification; controlled campaign and beneficiary management; and disbursements restricted by ownership, verification, balance and approval rules. The repository contains passing Hardhat and API test suites for these flows.

7.8 Discussion of Results

The implementation is strongest where application controls and blockchain proof complement each other. Proof makes a completed event tamper-evident; database transactions, RBAC and audit logs control operations. It does not prove an off-chain beneficiary outcome. Current limits are live credentials, production hardening, controlled user evaluation and performance testing. The hybrid risk module is integrated and tested, but remains disabled until the database migration, operational review process and configuration are ready. The CharityVault contract remains separate experimental work and is not part of the deployed payment flow.

7.9 System Screenshots

The final demonstration journey is sign-in, campaign browsing, payment, receipt/proof status and history. Administration evidence covers review, beneficiary verification, payout approval, audit records and exports. Screenshots should be captured from the final deployment immediately before submission so they cannot become stale.

CHAPTER EIGHT: CONCLUSION AND RECOMMENDATIONS

8.1 Summary

ChangiaTanzania applies familiar web and mobile-money-oriented donation workflows alongside blockchain proofs and administrative controls. The work covers the complete operational path from campaign publication to donation, receipt, proof verification, beneficiary management, controlled disbursement, audit record and export.

8.2 Achievement of Objectives

The project achieved secure role access through authentication and RBAC; donation management through payment sessions, idempotent callbacks and receipts; transparency through deterministic on-chain proofs and public lookup; accountability through beneficiary, balance and approval controls; and administrative support through dashboards, reports, notifications, audit records and an integrated risk-review capability. The risk feature remains disabled pending migration and operational activation.

8.3 Conclusion

The system shows that blockchain can be useful as a restrained proof mechanism rather than a replacement for normal payment and database infrastructure. Transparency improves when verifiable records are combined with privacy protection, operational auditability and separation of duties. The implementation is a credible foundation for NGO

donation management but requires formal operational rollout work before production financial use.

8.4 Recommendations

Before production deployment, the project should complete payment-provider certification, replace local file storage, introduce monitoring and backup procedures, perform independent security and load tests, and conduct structured end-user acceptance testing with retained consent and survey data.

8.5 Future Work

Future work should evaluate the hybrid fraud/risk module with sufficient real donation history and reviewed outcomes, consider the CharityVault escrow/multisignature contract only after governance review, move proofs to an appropriate production network, add durable object storage and expand accessibility and mobile support. Mainnet use should follow security audit, key-management and compliance preparation rather than being enabled solely because the code exists.

REFERENCES

- Alabdulkarim, L. et al. (2022). Blockchain-based donations traceability framework. *Journal of King Saud University - Computer and Information Sciences*, 34(10), 9442–9454. <https://doi.org/10.1016/j.jksuci.2022.09.021>
- Howson, P. (2021). Crypto-giving and surveillance philanthropy: Exploring the trade-offs in blockchain innovation for nonprofits. *Nonprofit Management and Leadership*, 31(4), 805–820. <https://doi.org/10.1002/nml.21452>
- Muneeb, M. et al. (2020). A framework to make charity collection transparent and auditable using blockchain technology. *Computers & Electrical Engineering*, 83, 106588. <https://doi.org/10.1016/j.compeleceng.2020.106588>
- Sung, J., Bock, G.-W., & Kim, H.-M. (2023). Effect of blockchain-based donation system on trustworthiness of NPOs. *Information & Management*, 60(5), 103812. <https://doi.org/10.1016/j.im.2023.103812>
- World Bank. (2017). *Tanzania economic update*. <https://documents1.worldbank.org/curated/en/340591491579331322/pdf/114125-NWP-P156957-PUBLIC-add-series-9th-TEU-April-7-2017-reduced.pdf>

APPENDICES

Appendix A: Questionnaire

A structured Google Forms questionnaire was shared through Tanzania Red Cross Society cooperation chat groups and among fellow students (Section 3.3) to gather early problem and preference themes ahead of design. It received 26 responses. Individual response rows were not retained; the instrument and the aggregated per-question summary below are reproduced from the retained chart export.

A.1 Instrument

Table A.1: Questionnaire instrument

#	Question	Response type
1	Do you trust current NGO donation systems?	Yes / No / Not Sure
2	Can you easily track how your donation is used after donating?	Yes / No / Sometimes
3	Do you think NGOs should provide transparent donation tracking?	Yes / No
4	Do you think blockchain can improve transparency in NGO donations?	Yes / No / Not Sure

#	Question	Response type
5	Do NGOs need better systems for managing donations?	Yes / Maybe / No

A.2 Aggregated Responses (n = 26)

Table A.2: Aggregated response summary

#	Question	Response breakdown
1	Do you trust current NGO donation systems?	No 42.3%, Not Sure 53.8%, Yes 3.9%
2	Can you easily track how your donation is used after donating?	No 61.5%, Sometimes 38.5%
3	Do you think NGOs should provide transparent donation tracking?	Yes 100%
4	Do you think blockchain can improve transparency in NGO donations?	Yes 76.9%, Not Sure 23.1%
5	Do NGOs need better systems for managing donations?	Yes 76.9%, Maybe 23.1%

These results are a convenience sample, not a representative survey, and are used as directional evidence for the design themes in Section 4.3: uncertain trust in existing systems, difficulty tracking a donation after payment, near-unanimous demand for transparent tracking, majority support for blockchain-backed verification, and majority demand for better donation management tooling.

Appendix B: Document Review Matrix

Document review was one of the three data-gathering methods (Section 3.7), alongside the questionnaire and observation of the developed application.

Table B.1: Document review matrix

Document / system	Source	Year	Scope	Finding taken
Effect of blockchain- based donation system on trustworthiness of NPOs	Sung, Bock & Kim, <i>Information & Management</i> (Sung et al., 2023)	2023	Blockchain donation platform, trust perception	Transparency, immutability and efficiency improve perceived nonprofit trustworthiness
Crypto-giving and surveillance philanthropy	Howson, <i>Nonprofit Management and Leadership</i> (Howson, 2021)	2021	Governance trade-offs in crypto-giving	Technology alone does not resolve governance; role separation and audit trails still needed
Blockchain- based donations traceability framework	Alabdulkarim et al., <i>J. King Saud Univ. CIS</i> (Alabdulkarim et al., 2022)	2022	Traceability framework design	Immutable event records and end-to-end traceability address uncertainty over fund use

Document / system	Source	Year	Scope	Finding taken
A framework to make charity collection transparent and auditable using blockchain	Muneeb et al., <i>Computers & Electrical Engineering</i> (Muneeb et al., 2020)	2020	Charity collection auditability	Cryptocurrency-first design creates an adoption mismatch for mobile-money users
Tanzania Economic Update	World Bank (World Bank, 2017)	2017	Financial inclusion, mobile money	Mobile money is a major contributor to financial inclusion in Tanzania, motivating fiat-first payment design
MINI 1 prior submission (NASIBU Y ISAKA 2106307225519.pdf)	Candidate's own earlier report	2026	Prior proposal for the same project	Retained: problem statement, questionnaire instrument and raw response chart export (reproduced in Appendix A)

Document / system	Source	Year	Scope	Finding taken
docs/BUSINESS_RULES.md	Internal architecture documentation	2026	Donation, disbursement and audit rules	Dual-approval threshold, beneficiary verification, immutability rules
docs/SECURITY.md	Internal architecture documentation	2026	Authentication and data protection	JWT/refresh rotation, bcrypt, RBAC, rate limiting design
docs/PAYMENT_ABSTRACT.md	Internal architecture documentation	2026	Payment provider abstraction	Provider- agnostic ses- sion/callback pattern behind a single service boundary
docs/BLOCKCHAIN_ARCHITECTURE.md	Internal architecture documentation	2026	Proof recording design	Backend-only chain access; deterministic proof hash; no PII on-chain
deployment/README.md	Internal deployment documentation	2026	Production hosting	Free-tier hosting topology reproduced in Appendix E

Appendix C: Source Code (selected excerpts)

Three excerpts illustrate the controls most central to the project's accountability claims: a recomputable proof hash, an idempotent payment callback, and the dual-approval disbursement rule.

C.1 Deterministic Proof Hash

server/src/services/blockchain.service.ts

```
/**
 * Deterministic proof hash (Decision 010): SHA-256 over donation ID, campaign
 * ID, amount, receipt number, payment reference and timestamp. Recomputable
 * from the donation row alone, so verification never needs extra storage.
 */
export function computeProofHash(input: {
  donationId: number
  campaignId: number
  amount: number
  receiptNumber: string
  paymentReference: string
  createdAt: Date
}): string {
  const payload = [
    input.donationId,
    input.campaignId,
    input.amount,
    input.receiptNumber,
    input.paymentReference,
    input.createdAt.toISOString(),
  ].join(':')
  return `0x${createHash('sha256').update(payload).digest('hex')}`
}
```

Because the hash is a pure function of the donation row, public verification (Section 6.11) never needs to store a second copy of the proof input; it recomputes the hash and compares it against the on-chain record.

C.2 Idempotent Payment Callback

server/src/services/payment.service.ts

```
/**
 * Handle a provider callback: verify authenticity, then either finalize the
 * donation or mark the attempt terminal. Idempotent, so duplicate callbacks
 * never double-record (flows/payment-flow.md).
 */
export async function handleCallback(
  payload: unknown,
  context?: CallbackContext,
): Promise<CallbackResult> {
  const provider = getPaymentProvider()
  const reference = provider.extractReference(payload)
  if (!reference) throw ApiError.badRequest('Invalid payment callback')

  const transaction = await findTransactionByReference(reference)
  if (!transaction) throw ApiError.notFound('Payment not found')

  const result = provider.verifyCallback(payload, transaction, context)
  if (!result.verified) throw ApiError.badRequest('Payment callback could not be

  return settleTransaction(transaction, reference, result)
}

// Both webhooks and the ClickPesa status recovery path call settleTransaction,
// so a delayed or duplicated webhook cannot double-credit a donor.
async function settleTransaction(transaction, reference, result) {
  if (transaction.status === 'success') {
```

```

    // Already finalized: return the existing donation without re-processing.
    return { status: 'success', reference, donationId: transaction.donationId ??
  }
  // ...creates the donation and payment_transactions rows atomically.
}

```

This guard is what makes payment finalisation safe under retried or duplicated gateway webhooks (Section 7.6, TC-02).

C.3 Dual-Approval Disbursement Rule

server/src/services/disbursement.service.ts

```

/**
 * A payout that keeps the campaign under the dual-approval threshold is
 * auto-approved and paid immediately; one that reaches it waits for a
 * separate administrator's decision (Decision 020).
 */
let requiresApproval: boolean
if (actor.role === 'admin') {
  // Administrators keep the per-payout rule (Decision 016).
  requiresApproval = input.amount >= DUAL_APPROVAL_THRESHOLD_TZS
} else {
  // Fundraisers are measured on the campaign's cumulative self-released total.
  const cumulative = await getCumulativeSelfReleased(input.campaignId)
  requiresApproval = cumulative + input.amount >= DUAL_APPROVAL_THRESHOLD_TZS
}
const autoApproved = !requiresApproval

```

The initiator and the approver are always different administrator accounts (Section 7.6, TC-06), so a single compromised or careless account cannot release a large payout unilaterally.

Appendix D: User Manual

Condensed from README.md and the role flows in flows/donor-flow.md, flows/admin-flow.md and docs/PRODUCT.md. All three roles share one sign-in screen; the application routes each role to its own console after login.

D.1 Donor

1. Open the landing page and browse active campaigns, or search and filter by category.
2. Open a campaign's details page to read its description, goal and raised amount.
3. Select **Donate**, registering or logging in first if not already signed in.
4. Enter an amount, choose a payment method (mobile money or bank), and review the donation.
5. Complete payment at the provider; on a verified callback the platform issues a receipt.
6. From **My Donations**, download the receipt, view donation history, and open **Verify** to see the donation's blockchain proof status.
7. The donor dashboard also shows Impact Points earned and the campaigns the donor supports.

D.2 Fundraiser

1. A donor applies to become a fundraiser; an administrator reviews the application (approve or reject with a reason).
2. Once approved, the fundraiser opens their console to create a campaign (title, description, category, goal, dates, image).
3. A new campaign starts in **Pending Review**; it is not publicly visible until an administrator approves it.
4. Once active, the fundraiser adds beneficiaries to the campaign; an administrator must verify each beneficiary before it can receive a payout.
5. The fundraiser initiates disbursements to verified beneficiaries. Payouts that keep the campaign's cumulative self-released total under the dual-approval threshold pay out immediately; payouts that reach the threshold wait for administrator approval.

6. The fundraiser dashboard shows raised totals, disbursed totals and stat tiles per campaign.

D.3 Administrator

1. Sign in and land on the admin dashboard, which summarises campaigns, donations and pending review queues.
2. Review fundraiser applications and fundraiser-submitted campaigns (approve or reject with a reason).
3. Manage beneficiaries: verify submitted beneficiaries before they become payout-eligible.
4. Manage users: change account status, or promote a user to administrator.
5. Approve or reject disbursements that reached the dual-approval threshold; an administrator can never approve their own initiated payout (Section 7.6, TC-06).
6. Open **Reports** for exports, **Audit Logs** for the immutable action history, and **Notifications** for system alerts.
7. When the integrated risk-review module is enabled, review the fraud/anomaly queue for unusual completed donations.

Appendix E: Installation Guide

Condensed from `README.md` (local development) and `deployment/RUNBOOK.md` (production deploy, reproduced in full there).

E.1 Local Development

Prerequisites: Node.js LTS and a PostgreSQL database (a free Neon instance works).

```
# Client (frontend) - http://localhost:5173
```

```
cd client
```

```
npm install
```

```
npm run dev
```

```
# Server (backend) - http://localhost:4000
```

```

cd server
npm install
cp .env.example .env    # set DATABASE_URL and secrets
npm run dev

# Contracts (blockchain, local Hardhat node)
cd contracts
npm install
npm run compile
npm test
npm run node             # local chain
npm run deploy:local     # deploy to local chain

```

Each package carries its own `.env.example`; `.env` files are never committed.

E.2 Production Deployment (summary)

The live deployment (Table E.1) follows the five ordered steps in `deployment/RUNBOOK.md`: deploy the proof contract to Sepolia, provision Neon PostgreSQL and seed the first administrator, push to GitHub, deploy the backend on Render from `render.yaml`, and deploy the frontend on Vercel with `VITE_API_BASE_URL` pointed at the Render URL. `CLIENT_ORIGIN` on the backend must then be set to the exact Vercel URL for cross-site cookies to work.

Table E.1: Production hosting topology

Tier	Host	Notes
Frontend	Vercel (static)	Root directory client; SPA rewrite via <code>client/vercel.json</code>
Backend	Render (Node web service)	Free tier sleeps after ~15 min idle; first request then takes 30 to 60 seconds

Tier	Host	Notes
Database	Neon (managed PostgreSQL)	Same instance as local development
Blockchain	Ethereum Sepolia	Contract address recorded in the runbook

Appendix F: Test Cases

Combines the API/workflow test cases summarised in Section 7.6, the Hardhat smart-contract unit tests, and the manual cross-browser and responsive checklist. All automated cases pass as of the latest test run.

F.1 API and Workflow Test Cases

Table F.1: API and workflow test cases

ID	Description	Precondition	Steps	Expected	
				Result	Status
TC-01	Registration	None	Submit valid registration details	Account and session created	Pass
TC-02	Duplicate callback	A pending payment session exists	Send the same provider callback twice	Exactly one donation and one credit; second call is a no-op	Pass

ID	Description	Precondition	Steps	Expected	
				Result	Status
TC-03	Public proof lookup	A completed donation exists	Look up the donation by receipt number	Status returned without donor identity or payment reference	Pass
TC-04	Campaign review	Fundraiser submits a campaign	Fundraiser saves a new campaign	Campaign is not publicly visible before administrator approval	Pass
TC-05	Beneficiary control	Beneficiary not yet verified	Fundraiser initiates a payout to that beneficiary	Payout request is rejected	Pass
TC-06	Separation of duties	Disbursement pending approval, initiated by admin A	Admin A attempts to approve their own disbursement	Approval is rejected	Pass
TC-07	Balance control	Requested amount exceeds campaign's available balance	Initiate a disbursement above the balance	Request is rejected	Pass

ID	Description	Precondition	Steps	Expected	
				Result	Status
TC-08	Duplicate proof	A proof already exists on-chain for an identifier	Attempt to register the same identifier again	Contract reverts with ProofAlreadyExists	Pass

F.2 Smart Contract Unit Tests

contracts/test/TransparencyRegistry.test.ts (Hardhat, Chai), 4 tests:

Table F.2: Smart contract unit tests

Test	Assertion	Status
Records and verifies a donation proof	verifyDonation returns true; donationCount increments; stored campaign ID and proof hash match the input	Pass
Rejects duplicate donation proofs	Registering the same donation ID twice reverts with ProofAlreadyExists	Pass
Blocks a non-owner from recording proofs	A non-owner call to registerDonation reverts with NotOwner	Pass
Records donation and disbursement independently	A donation proof and a disbursement proof for the same campaign are stored and verified separately	Pass

F.3 Manual and Responsive Testing Checklist

Source: `testing/manual-testing.md`. Executed against the local build before each deployment.

Table F.3: Manual and responsive testing checklist

Area	Checks
Authentication	Register, login, logout, forgot password
Campaigns	Create, publish, edit, delete
Donations	Mobile money, bank, receipt verification, blockchain proof verification
Dashboard	Statistics, reports, notifications
Beneficiaries	Add, edit, link to campaign
Responsive	Desktop, tablet, mobile breakpoints
Browser	Chrome, Edge, Firefox

Appendix G: Gantt Chart

Derived directly from the repository's commit history (`git log`), not re-estimated after the fact. Each stage's date is the day its milestone commit (for example `docs: mark Stage 6 administrator complete`) landed.

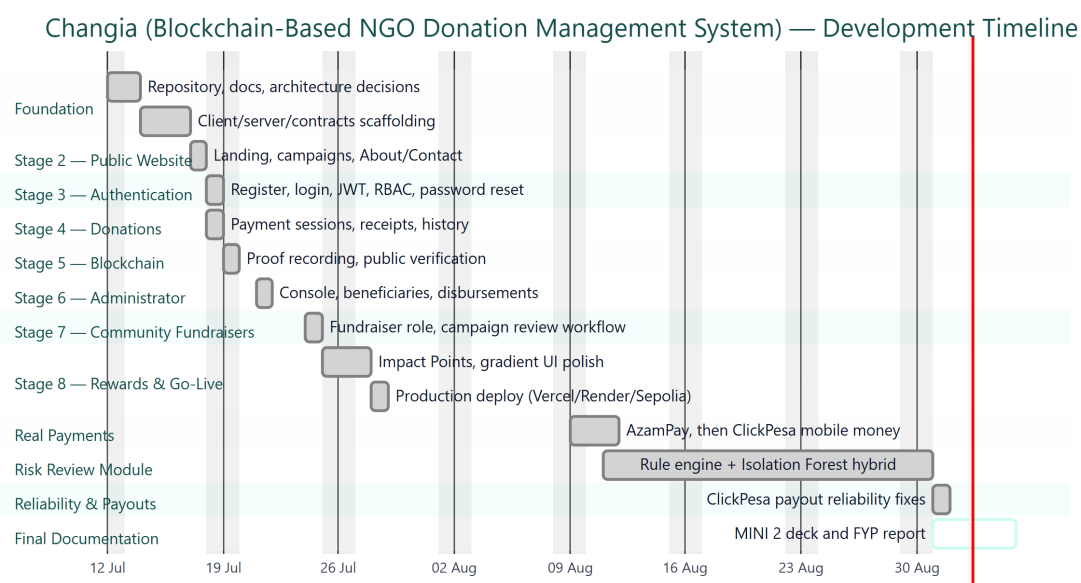


Figure G.1: Project development timeline

Table G.1: Development phases by date

Phase	Dates (2026)
Foundation (scaffolding, architecture)	12 Jul to 17 Jul
Stage 2: Public website	17 Jul
Stage 3: Authentication	18 Jul
Stage 4: Donations	18 Jul
Stage 5: Blockchain proof	19 Jul
Stage 6: Administrator console	21 Jul
Stage 7: Community fundraisers	24 Jul to 25 Jul
Stage 8: Impact Points and go-live	25 Jul to 29 Jul
Real payments (AzamPay, then ClickPesa)	9 Aug to 12 Aug
Integrated risk-review module	11 Aug to 31 Aug
Reliability and payout fixes	31 Aug to 1 Sep
Final documentation (this report, MINI 2 deck)	31 Aug to submission

Appendix H: Budget

The deployed system uses free tiers throughout; the only recurring cost the project would add is an optional custom domain.

Table H.1: Project budget

Item	Provider	Tier used	Monthly cost
Frontend hosting	Vercel	Free (Hobby)	TZS 0
Backend hosting	Render	Free (sleeps after ~15 min idle)	TZS 0
Database	Neon PostgreSQL	Free	TZS 0
Blockchain RPC	Alchemy / public Sepolia RPC	Free	TZS 0
Test ETH (Sepolia)	Public faucet	Free	TZS 0
Domain name	Optional, not purchased	—	~USD 12/year if added
Backend always-on upgrade	Render Starter (optional)	Paid	~USD 7/month if added

Developer time is not costed, as this is an academic final year project rather than a commissioned engagement.

Appendix I: Additional Screenshots

Pending, consistent with Section 7.9: screenshots are captured from the live deployment (<https://changia-hazel.vercel.app>) immediately before submission so they reflect the exact system evaluated, rather than an earlier state that could go stale before the viva.

Appendix J: Full Entity Relationship Diagram

The core-entity diagram in Section 6.4.1 omits supporting tables for readability. All 14 tables are shown below.

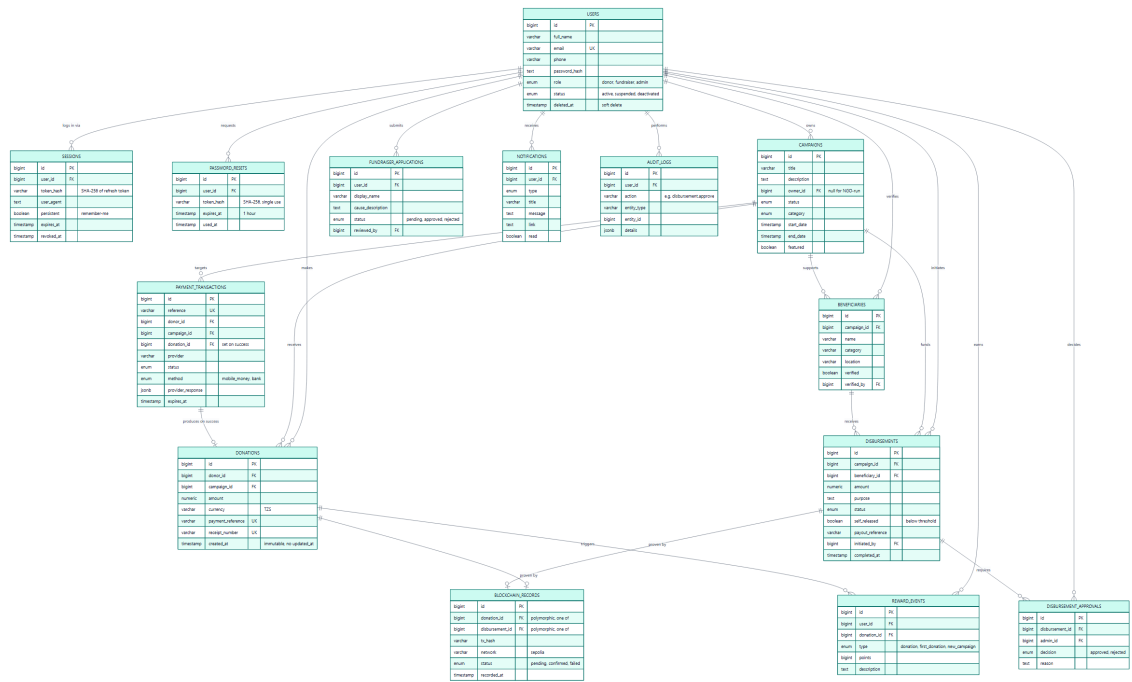


Figure J.1: Full entity relationship diagram, all 14 tables